

UNIT – 10

File Handling

C Programming (CSC115)
BSc CSIT First Semester (TU)

Prepared by:
Dabbal Singh Mahara
ASCOL (2025)

Console I/O

- Programs without data files accept input data from the keyboard at the time of execution and write output to the monitor. This type of I/O is called console I/O
- The main limitation of this approach is that it is very difficult to handle a large volume of data.
- The I/O functions like `printf()`, `scanf()`, `getchar()`, `putchar()`, `gets()`, `puts()` are console oriented I/O functions.
- Console oriented functions use keyboard as input device and monitor as output device.

Problems with console I/O

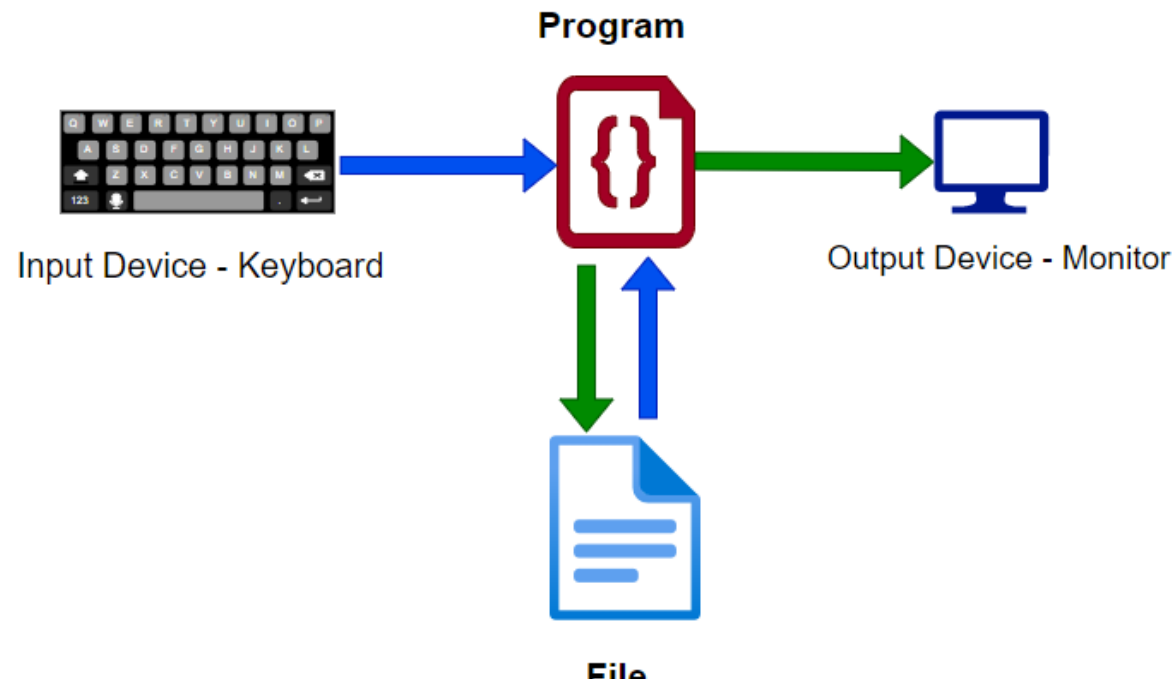
- Entire data is lost when either the program is terminated or the computer is turned off.
- When the volume of data to be entered is large, it takes a lot of time to enter the data.
- If user makes a mistake while entering data, whole data has to be re-entered.

File I/O

- Many applications require that information be written to or read from the auxiliary memory.
- Such type of I/O in which writing and reading of the data involves secondary storage in the form of data files is known as file I/O.
- A collection of data which is stored on a secondary device like a hard disk is known as **a file**.
- A file is generally used as real-life applications that contain a large amount of data.
- The data file allows us to store information permanently and to access and alter that information whenever necessary.

Streams

- Conceptually, the C program deals with a stream instead of directly with a file.
- A stream is a logical connection between application program and a file through which we can transfer the data either for storing or reading.



Input Stream

- A program uses an *input stream* to read data from a source, one item at a time.

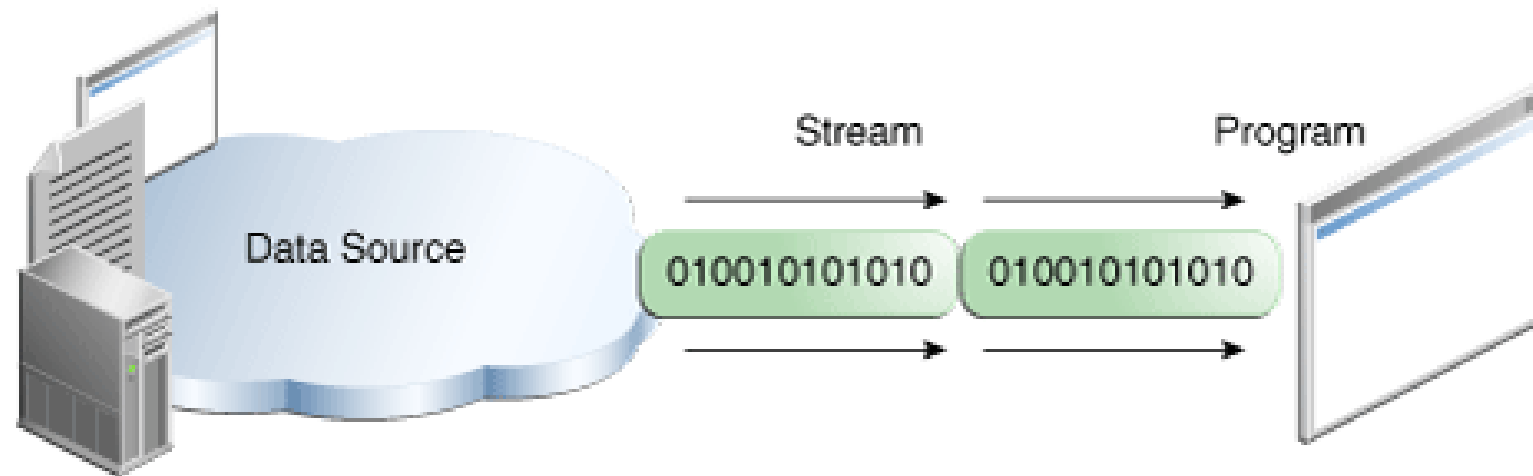


Figure: Input Stream – reading information into a program

Output Stream

- A program uses an *output stream* to write data to a destination, one item at time.

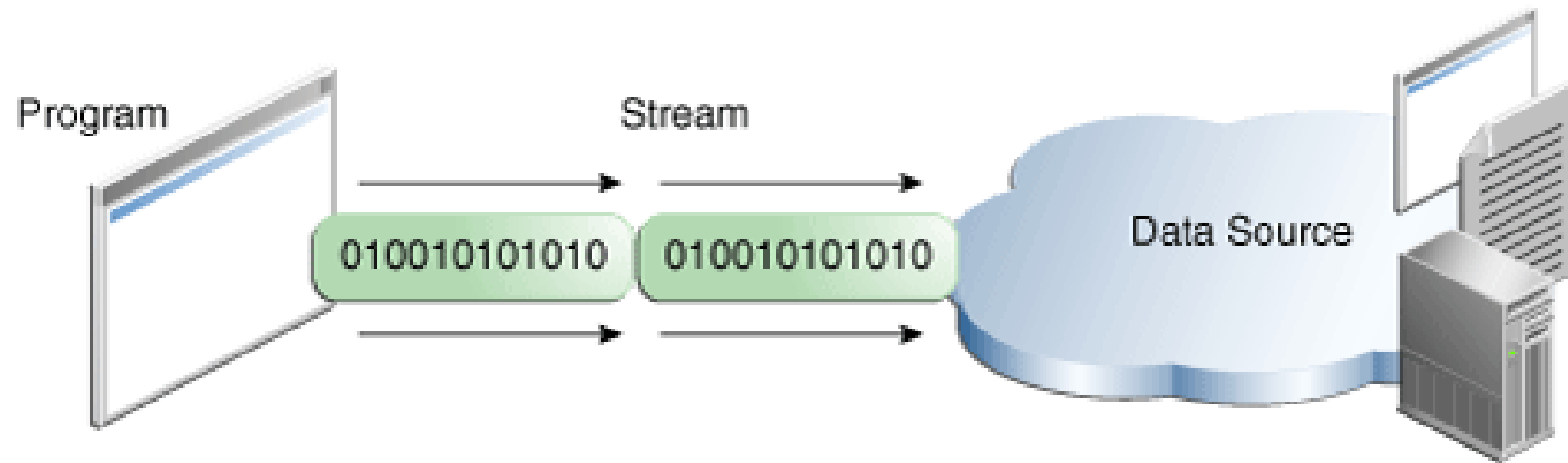
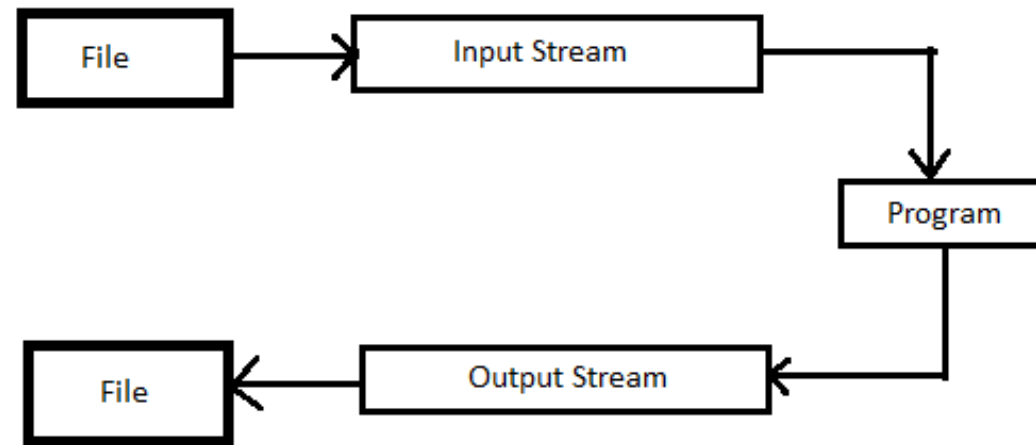


Figure: Output Stream - writing information from a program.

File Stream

- The type of the C data structure that represents a stream is called FILE.
- A FILE object holds all of the internal state information about the connection to the associated file, including such things as the file position indicator and buffering information.
- Opening a file with the fopen function creates a new stream and establishes a connection between the stream and a file. This may involve creating a new file.



Data files

- Stream oriented data files are **two** types.
- In the **first** category, the data file comprises consecutive characters. These characters can be interpreted as individual data items or as components of strings or numbers. These are called **text files**.
- In the **second** category, often called as **unformatted data files**, organizes data into blocks containing contiguous bytes of information. These blocks represent more complex data structures, such as arrays and structures. These files are called **binary files**

I. Text files

- A **text file** stores data in the form of alphabets, digits and other special symbols by storing their ASCII values and are in a human readable format. For example, any file with a .txt, .c, etc extension.
- When you open those files, you'll see all the contents within the file as plain text. You can easily edit or delete the contents.
- They take minimum effort to maintain, are easily readable, and provide the least security and takes bigger storage space.

Binary files

- A **binary file** contains a sequence or a collection of bytes (binary form: 0's and 1's) which are not in a human readable format.
- For example, files with .exe, .com, .mp3 extension.
- Since the data is not human readable it also adds to the security of the content as one might not be able to get data if the structure is not known.

Difference between Text and Binary Files

- A **newline** character in a text file is first converted into a carriage return-linefeed combination and then written to the disk. Similarly, when read by a **text file** the carriage return-linefeed combination is converted into a **newline**. However, in a **binary file**, no such conversions take place.
- The texts and the characters are stored one character per byte as it should be (char occupies 1 byte in the memory) . For example, we have a number 567392. According to integer storage convention, it should occupy 4 bytes in the disk but it does not. It occupies 6 bytes, i.e., 1 byte for every digit in the number. Also, the number 56.9057 will occupy 7 bytes in the disk.
- But binary files store the data in the binary form which shall use only 4 bytes for storing the integer and floating point data.
- In the **text mode**, a special character with the ASCII code 26 is inserted at the end of the file. This character when encountered returns the EOF signal to the program. But, in the **binary mode**, we do not have any special character to signify the EOF.

Defining and Opening a file

- A program must open a file before to perform reading and writing operations on it.
- Opening a file establishes a link between the program and the operating system.
- To use a file four essential actions should be carried out. These are,
 - a. Declare a file pointer variable.
 - b. Open a file using the `fopen()` function.
 - c. Process the file using suitable functions.
 - d. Close the file using the `fclose()` function.

Declaration of file pointer

- When working with files, you need to declare a pointer of type FILE.
- FILE is a special structure defined in <stdio.h>.
- The members of the FILE structure are used by the program in various file access operation.
- FILE *file_pointer;
Eg : FILE *fp;
- FILE establishes a buffer area where data is temporarily stored while being transferred between computer memory and the data file.
- file_pointer variable indicates the beginning address of buffer area. This pointer is often referred to as a stream pointer, file pointer or simply stream.

Opening a file

- Opening a file is performed using the `fopen()` function defined in the `stdio.h` header file.

- The syntax for opening a file in standard I/O is:

```
fptr = fopen("filename","mode");
```

Here, `fptr` is the FILE pointer (`FILE *fptr`), which will hold the reference to the opened file, `filename` is the name of the file to be opened and `mode` specifies the purpose of opening the file.

- When `fopen()` function opens a file in memory, it returns a pointer to this particular file. If `fopen()` function can't open the file then it will return `NULL`.

- For example,

```
ptr1 = fopen("E:\\cprogram\\newprogram.txt","w");
```

```
ptr2 = fopen("E:\\cprogram\\oldprogram.bin","rb");
```

File Opening Modes

- File opening modes specify the way in which a file should be opened
 - i.e. for reading, writing, or both, appending at the end of the file etc.
- The modes are denoted with:
 - "r" : for read mode
 - "w" : for write mode
 - "a" : for append mode
 - "r+" : for read and write mode
 - "w+" : for write and read mode
 - "a+" : for append and read mode

File Opening Modes

mode	description
r	opens a text file in reading mode
w	opens or create a text file in writing mode.
a	opens a text file in append mode
r+	opens a text file in both reading and writing mode
w+	opens a text file in both reading and writing mode
a+	opens a text file in both reading and writing mode
rb	opens a binary file in reading mode
wb	opens or create a binary file in writing mode
ab	opens a binary file in append mode
rb+	opens a binary file in both reading and writing mode
wb+	opens a binary file in both reading and writing mode
ab+	opens a binary file in both reading and writing mode

Opening a file in read mode

- `FILE *ptr;`
`ptr = fopen("student.txt", "r");`
- This mode opens an existing file for reading only purpose.
- If file exists, then the file is loaded into memory and a pointer that points to the first character in the file is assigned to the file pointer.
- If file does not exist, then it returns `NULL`.

Opening a text file in write mode

- `FILE *ptr;`
`ptr = fopen("filename", "w");`
- This mode opens a file for writing purpose only.
- If file exists, then the contents of the file are overwritten.
- If the file does not exist, a new file is created.
- File pointer ptr is at the beginning of file.
- It returns NULL, it is unable to open the file in write mode.

Opening a text file in append mode

- This mode opens an existing file for appending purpose i.e. adding new data at the end of file.
- If file exists, it loads the file into memory and a pointer that points last character of the file is assigned to the file pointer.
- If the file does not exist, a new file is created.
- It returns NULL, it is unable to open the file in append mode.
- `FILE *ptr;`
`ptr = fopen("filename", "a");`

Closing a File

- A file must be closed as soon as all operations on it have been completed.
- The closing a file ensures that all outstanding data associated with the file is flushed out from the buffers and all links to the file are broken.
- It also prevents any accidental misuse of the file.
- In cases where there is a limit to the number of files that can be kept open simultaneously, closing unwanted files help in opening the required ones.
- Another instance where we have to close a file is when we want to reopen the same file in another mode.
- The `fclose()` function is used to close an already opened file.
- General Syntax : `fclose( fp );`
- Here `fclose()` function closes the file and returns zero on success, or EOF if there is an error in closing the file.
- This EOF is a constant defined in the header file `stdio.h`.

File I/O Functions

- Once a file is opened, reading out of or writing to it is accomplished using the standard I/O functions.
- There are a number of File I/O functions available to use.
 1. Character I/O functions
 2. String I/O functions
 3. Formatted I/O functions
 4. Block I/O functions

Character I/O Functions

- Two functions `fgetc()` and `fputc()` are the simplest functions which can be used to read and write individual characters to a file.
- `fgetc()` is used to read a character from a file
 - Syntax: `char_var = fgetc(fp);`
- `fputc()` is used to write a character to a file.
 - Syntax : `fputc('character' or char_var, fp);`

Example: fputc() and fgetc()

```
#include <stdio.h>

int main()
{

    FILE *fp;
    fp = fopen("file1.txt", "w");//opening file
    fputc('a',fp);//writing single character into file
    fclose(fp);//closing file
    return 0;
}
```

```
#include<stdio.h>

int main(){
FILE *fp;
char c;
fp=fopen("myfile.txt","r");

while((c=fgetc(fp))!=EOF) {
    printf("%c",c);
}
fclose(fp);
return 0;
}
```


Example: 2 Character I/O to file

```
#include<stdio.h>

int main()
{
    FILE *fp;
    char ch;
    fp = fopen("myfile.txt", "w");
    printf("Enter text until Enter key: \n");
    while( (ch = getchar()) != '\n') {
        fputc(ch, fp);
    }
    fclose(fp);
    return 0;
}
```

```
#include<stdio.h>

int main()
{
    FILE *fp;
    char ch;
    fp = fopen("myfile.txt", "r");
    while( (ch = fgetc(fp)) != EOF)
        putchar(ch);
    fclose(fp);
    return 0;
}
```

Detecting the end of a file

- EOF is a special character an integer with ASCII value 26 that indicates the end of file has been reached.
- This character can be generated from the keyboard by typing CTRL + Z
- The symbolic constant EOF is defined in stdio.h.
- When we are creating a file, the special character EOF is inserted after the last character of the file by the OS.
- When reading from a text-mode file character by character, one can look for the end-of-file character.
- Eg: `while((ch = fgetc(fp))!=EOF).`
- This is the general representation of the End Of the File.

String I/O Functions

- Using string I/O functions `fgets()` and `fputs()`, data can be read from a file or written to a file in the form of array of characters.
- `fgets()`: is used to read string from file
 - Syntax: `fgets(string, int_value, fp);`
 - Here, `int_value` is the number of characters in the string
- `fputs()`: is used to write string to a file
 - Syntax: `fputs(string, fp);`

Example: String I/O to file

```
#include<stdio.h>

int main()
{
    FILE *fp;
    char str[100];
    fp = fopen("hello.txt", "w");
    printf("Enter text: \n");
    gets(str);
    fputs(str, fp);
    fclose(fp);
    return 0;
}
```

```
#include<stdio.h>

int main()
{
    FILE *fp;
    char str[100];
    fp = fopen("hello.txt", "r");
    fgets(str,20,fp);
    printf("\n %s", str);
    fclose(fp);
    return 0;
}
```

Appending a string to a file

```
#include<stdio.h>
int main()
{
    FILE *fp;
    char str[100];
    fp = fopen("hello.txt", "a");
    printf("Enter text: \n");
    gets(str);
    fputs(str, fp);
    fclose(fp);
    return 0;
}
```

Write a program to copy contents of a file to another file.

```
#include<stdio.h>
int main()
{
    FILE *fp1, *fp2;
    char ch;
    fp1 = fopen("file1.txt", "r");
    fp2 = fopen("file2.txt", "w");

    while( (ch = fgetc(fp1)) != EOF) {
        fputc(ch, fp2);
    }
    printf("\n File copied ...");
    fclose(fp1);
    fclose(fp2);
    return 0;
}
```

Given a text file copy the contents of the file deleting all the vowels.

```
#include<stdio.h>
int main()
{
    FILE *fp1, *fp2;
    char ch;
    fp1 = fopen("file1.txt", "r");
    fp2 = fopen("file2.txt", "w");

    while( (ch = fgetc(fp1)) != EOF) {
        if(ch != 'a' && ch != 'e' && ch != 'i' && ch != 'o' && ch != 'u')
            fputc(ch, fp2);
    }
    printf("\n File copied ...");
    fclose(fp2);
    return 0;
}
```

Formatted I/O Functions

- `fprintf()` and `fscanf()` are two formatted I/O functions used to write numbers, characters or strings to a file or read from the file according to our requirement format.
- `fprintf()`: is used to write integer, float, char or string value to a file
 - Syntax: `fprintf(fp, "control_string", list_of_variables);`
- `fscanf()`: is used to read integer, float, char or string value from a file
 - Syntax: `fscanf(fp, "control_string", &list_of_variables);`

Example:

Program to create file named employee.txt and write name, id, address and salary of a employee to this file.

```
#include<stdio.h>
int main()
{
    FILE *fp;
    char name[20];
    int id;
    char address[40];
    float salary;
    fp = fopen("E:\\employee.txt", "w");
    printf("\n Enter name employee:\t");
    gets(name);
```

```
    printf("\n Enter Id: \t");
    scanf("%d", &id);
    fflush(stdin);
    printf("\n Enter Address: \t");
    gets(address);
    printf("\n Enter Salary: \t");
    scanf("%f", &salary);
    fprintf(fp, "%s\t %d\t %s \t %f", name,
    id, address, salary);
    fclose(fp);
    return 0;
}
```

Use of fflush() function

- fflush() : is used to flush out any data remaining in the buffer.
- The argument to fflush() must be the buffer which we want to flush out. Eg. stdin, which means buffer related to standard input device i.e. keyboard

Reading and Writing to File using fprintf() and fscanf()

```
#include<stdio.h>

struct emp
{
    char name[10];
    int age;
};

int main() {
    struct emp e;
    char filename[20];
    FILE *p,*q;
    printf("Enter file name: ");
    gets(filename);
    p = fopen(filename, "w");
```

```
    for(i=0;i<5;i++ ) {
        printf("Enter Name and Age:");
        scanf("%s %d", e.name, &e.age);
        fprintf(p, "\n%s \t %d", e.name, e.age);
    }
    fclose(p);
    q = fopen("one.txt", "r");
    do {
        fscanf(q, "%s %d", e.name, &e.age);
        printf("%s %d", e.name, e.age);
    }
    while(!feof(q));
    return 0;
}
```

Reading and Writing in a Binary File

- A binary file is the one where data is stored as collection of bytes in binary form not in ASCII character format.
- These files are not human readable and can be created and read only by specific programs written for them.
- Character I/O and string I/O functions allow reading/ writing of characters only, while the formatted I/O functions allow reading/writing of character data and numeric data both.
- But numbers are always stored as a sequence of characters using above I/O functions irrespective of text mode or binary mode. So, they occupy a lot of disk space.
- To solve this problem, record or block I/O is used with two functions: `fread()` and `fwrite()`.
- Record I/O writes numbers to the file in binary format so that numbers occupy space as usual such as integers 4 bytes, float 4 bytes, double 8 bytes etc.

Writing to a binary file

- To write into a binary file, we use the `fwrite()` function.
- It is used to write a block of data at a single attempt
- The functions take four arguments:
 - address of data to be written in the disk
 - size of data to be written in the disk
 - number of such type of data
 - pointer to the file where you want to write.
- `fwrite(address_of_Data, sizeData, numbers_of_Data, pointerToFile)`

`fwrite()` function

- Syntax: `size_t fwrite(const void *ptr, size_t size, size_t n, FILE *fp);`
- The `fwrite()` function writes the data specified by the void pointer `ptr` to the file.
 - `ptr`: it points to the block of memory which contains the data items to be written.
 - `size`: It specifies the number of bytes of each item to be written.
 - `n`: It is the number of items to be written.
 - `fp`: It is a pointer to the file where data items will be written.
- On success, it returns the count of the number of items successfully written to the file.
- On error, it returns a number less than `n`.

Writing to a binary file

```
#include <stdio.h>
#include <stdlib.h>
struct employee
{
    char name[20];
    int age;
    float salary;
};

int main()
{
    int n;
    struct employee emp;
    FILE *fptr;
    fptr = fopen("C:\\employee.txt","wb");
    if(fptr== NULL){
        printf("Error! opening file");
        // Program exits if the file pointer returns NULL.
        exit(1);
    }
```

```
do {
    printf("\n Enter name, age and salary of employee:\t");
    scanf("%s %d %f", emp.name, &emp.age, &emp.salary);
    fwrite(&emp, sizeof(emp), 1, fptr);
    printf("\n Add another record (Y/N) : ");
    fflush(stdin);
    ch = getche();
    } while ( ch == 'Y');
    fclose(fptr);

    return 0;
}
```

Reading from binary file

- fread() function is commonly used to read binary data.
- It accepts the same arguments as fwrite()
- Syntax:

`size_t fread(void *ptr, size_t size, size_t n, FILE *fp);`

- The ptr is the starting address of the memory block where data will be stored after reading from the file.
- The function reads n items from the file where each item occupies the number of bytes specified in the second argument.
- On success, it reads n items from the file and returns n.
- On error or end of the file, it returns a number less than n.

Reading from a binary file

```
#include <stdio.h>
#include <stdlib.h>
struct employee
{
    char name[20];
    int age;
    float salary;
};

int main()
{
    int n;
    struct employee emp;
    FILE *fptr;
    fptr = fopen("C:\\employee.txt","rb");
    if(fptr== NULL){
        printf("Error! opening file");
        // Program exits if the file pointer returns NULL.
        exit(1);
    }
```

```
do {
    printf("\n The employee records are: ");
    While( !feof(fp)) {
        fread(&emp, sizeof(emp), 1, fptr);
        printf("\n %s \t %d \t %f ", emp.name,
            emp.age, emp.salary);
    }
    fclose(fptr);

    return 0;
}
```

feof() function

- feof() function is a file handling function in C programming language which is used to find the end of a file.
- Declaration: `int feof(FILE *fp)`
- This function returns a non-zero value when End-of-File indicator associated with the stream is set, else zero is returned.

Example: feof()

```
#include <stdio.h>
```

```
int main () {  
    FILE *fp;  
    int c;  
  
    fp = fopen("file.txt","r");  
    if(fp == NULL) {  
        perror("Error in opening file");  
        return(-1);  
    }  
}
```

```
while(1) {  
    c = fgetc(fp);  
    if( feof(fp) ) {  
        break ;  
    }  
    printf("%c", c);  
}  
fclose(fp);  
  
return(0);  
}
```

Sequential vs Random Access to a File

- We can access the data stored in file in two ways, sequentially or random.
- Till now we were reading and writing from/to a file sequentially.
- While reading data from a file, the data items are read from the beginning of the file in sequence until the end of the file.
- Also while writing the data to a file, the data items are placed one after the other in a sequence.
- This is called sequential access.
- But we may need to access a particular data item placed in any location without starting from the beginning. This is called random access.
- Random access takes less time than the sequential access.

Random Access to a File

- There is no need to read each record sequentially, if we want to access a particular record.
- We can move file pointer back and forth according to our need.
- C supports these functions for random access file processing.
 - `fseek()`
 - `ftell()`
 - `rewind()`
- These three functions are called file positioning functions.
- The required header for these functions is:
`#include <stdio.h>`

fseek()

- If you have many records inside a file and need to access a record at a specific position, you need to loop through all the records before it to get the record.
- It will be time consuming to perform sequential access. Better option is `fseek()`.
- The `fseek()` function is used to set the file pointer to the specified offset.
- **Syntax:**

int `fseek`(**FILE** *`fptr`, **long int** `offset`, **int** `position`)

fseek() function

- The first parameter `fptr` is the pointer to the file.
- The second parameter `offset` is the number of positions(bytes) to be moved from the location specified by the third parameter.
- The third parameter `position` specifies the location where the offset starts.
- Third parameter `position` can be one of the following three constants:
 - `SEEK_SET` The beginning of the file.
 - `SEEK_END` The end of the file.
 - `SEEK_CUR` The current location of the cursor in the file.

Example: fseek()

```
#include <stdio.h>
```

```
int main()  
{
```

```
    FILE *fp;
```

```
    fp = fopen("test.txt", "r");
```

```
    // Printing position of pointer  
    printf("%ld", ftell(fp));
```

```
    // Moving pointer to 10 bytes from the beginning  
    fseek(fp, 10, SEEK_SET);  
    printf("%ld", ftell(fp));
```

```
    fseek(fp, 10, SEEK_END);  
    printf("%ld", ftell(fp));
```

```
    return 0;
```

```
}
```


ftell() function

- ftell() returns the current position of the file pointer within the file with respect to the starting of the file.
- Syntax: `long int ftell(FILE *fptr);`
- We can use ftell() function to get the total size of a file after moving file pointer at the end of file.
- We can use `SEEK_END` constant to move the file pointer at the end of file.

Example : ftell()

```
#include <stdio.h>
#include <conio.h>
int main () {
    FILE *fp;
    int length;
    fp = fopen("file.txt", "r");
    fseek(fp, 0, SEEK_END);
    length = ftell(fp);
    fclose(fp);
    printf("Size of file: %d bytes", length);
    getch();
}
```

rewind() function

- `rewind()`: It moves the control to beginning of the file.
- Declaration: `void rewind(FILE *fptr);`

rewind(): Example

```
#include<stdio.h>
#include<conio.h>
int main(){
FILE *fp;
char c;

fp=fopen("myfile.txt","r");

while((c=fgetc(fp))!=EOF){
printf(" %c",c);
}
```

rewind(fp); //moves the file pointer at beginning of the file

```
while((c=fgetc(fp))!=EOF){
    printf(" %c",c);
}
```

```
fclose(fp);
getch();
}
```



Thank you !